



## ΕΡΓΑΣΤΗΡΙΟ 1

### Εντολή Επανάληψης while

#### Στόχος Εργαστηρίου

Στόχος αυτού του εργαστηρίου είναι η εξοικείωση των φοιτητών με τη γλώσσα Python και πιο συγκεκριμένα:

- Τον τρόπο λειτουργίας των επαναλήψεων
- Εντολή επανάληψης while

#### Εντολή Επανάληψης while

Την εντολή επανάληψης while την συναντούμε σε όλες τις γλώσσες προγραμματισμού. Με την εντολή while μπορούμε να επαναλάβουμε την ομάδα εντολών που βρίσκονται από κάτω (μέσα σε κατακόρυφη στοίχιση – indentation) για όσο διάστημα είναι αληθής η λογική έκφραση μέσα στην παρένθεση. Γενική σύνταξη της εντολής while:

```
# The while loop
while (condition):
    (conditional command 1)
    (conditional command 2)
    (conditional command ...)
    (conditional command i)
```

Ας δούμε καλύτερα στην πράξη ένα πρόγραμμα που υπολογίζει τους όρους της σειράς Φιμπονάτσι που είναι μικρότεροι από 50 με χρήση του while.

Στα Μαθηματικά, οι Αριθμοί Φιμπονάτσι είναι οι αριθμοί της παρακάτω ακέραιης ακολουθίας:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...

Εξ ορισμού, οι πρώτοι δύο αριθμοί Φιμπονάτσι είναι το 0 και το 1, και κάθε επόμενος αριθμός είναι το άθροισμα των δύο προηγούμενων.

```
a, b = 0, 1

print(a, end=' ')
# Όσο το b είναι μικρότερο από 50 θα εκτελούνται οι πιο κάτω εντολές
while b < 50:
    print(b, end=' ') # Τυπώνει ένα κενό αντί για αλλαγή γραμμής μετά από κάθε
    εκτύπωση
    a, b = b, a + b

Result: 0 1 1 2 3 5 8 13 21 34
```

Στο πιο πάνω παράδειγμα αρχικοποιούμε τις μεταβλητές a και b με τις τιμές 0, 1 αντίστοιχα. Η μεταβλητή a περιέχει τον αριθμό Φιμπονάτσι που πρέπει να εκτυπώσουμε και η μεταβλητή b τον αμέσως επόμενο. Ακολουθεί η εντολή while στο σώμα της οποίας βρίσκεται ουσιαστικά όλη η ουσία του αλγορίθμου.



Στη συνθήκη του `while` ορίζουμε σαν όριο τον αριθμό 50 αφού ζητούμε τους αριθμούς της σειράς Φιμπονάτσι που είναι μικρότεροι του 50. Έτσι όσο η μεταβλητή `a` (ο αριθμός Φιμπονάτσι που πρέπει να εκτυπώσουμε) είναι μικρότερη από το όριο αυτό, εκτελείτε το σώμα του `while`! Δηλαδή:

1. Τυπώνεται η μεταβλητή `a`
2. Γίνονται δύο ταυτόχρονες αναθέσεις τιμών:
  - a. Η μεταβλητή `a` παίρνει την τιμή της μεταβλητής `b`
  - b. Η μεταβλητή `b` παίρνει την τιμή `a + b`

**Προσοχή:** Οι τιμές των μεταβλητών αλλάζουν *πραγματικά* από την αμέσως επόμενη εντολή.

Έτσι, στο παράδειγμα μας τα αποτελέσματα της εντολής `a, b = b, a + b` θα ισχύσουν από την επόμενη γραμμή κώδικα. Η μεταβλητή `a` θα πάρει την τιμή `b` αφότου τελειώσει η εντολή `a, b = b, a + b`. Έτσι για την ανάθεση τιμής στην `b` (που βρίσκεται στην ίδια εντολή) η `a` έχει ακόμη την παλιά της τιμή.

Εναλλακτικά για να επιτύχουμε το ίδιο αποτέλεσμα θα έπρεπε να αποθηκεύσουμε την τιμή της μεταβλητής `a` σε μια προσωρινή μεταβλητή, `c` για παράδειγμα, και για τον υπολογισμό της μεταβλητής `b` να χρησιμοποιήσουμε τη μεταβλητή αυτή, δηλαδή, `b = c + b`.

Την πρώτη φορά που εισερχόμαστε στο `while` η μεταβλητή `a` έχει την τιμή 0 και η `b` την τιμή 1. Αφότου τυπωθεί 0 (η τιμή της `a` δηλαδή) η μεταβλητή `a` θα πάρει την τιμή της μεταβλητής `b`. Στην ίδια εντολή (δηλαδή την ίδια στιγμή) η μεταβλητή `b` θα πάρει την τιμή του αθροίσματος `a + b` (δηλαδή `0 + 1` αφού η ανάθεση της τιμής στην `a` δεν έχει ισχύ ακόμη εφόσον δεν άλλαξε η εντολή). Έτσι μετά την εκτέλεση της ανάθεσης, η μεταβλητή `a` έχει γίνει 1 και η `b` `0+1` άρα 1.

Για όσους προβληματίστηκαν με την εντολή `print(b, end=' ')`, το `end=' '` ουσιαστικά επιβάλλει τις τιμές να εκτυπώνονται η μία δίπλα στην άλλη διαχωριζόμενες από τον κενό χαρακτήρα. Εννοείται πως ο χαρακτήρας αυτός μπορεί να είναι οποιοσδήποτε χαρακτήρας θελήσετε!

Ένα δεύτερο παράδειγμα είναι ένα πρόγραμμα το οποίο να εκτυπώνει κάθε φορά αυτό που εισάγει ο χρήστης μέχρι αυτός να γράψει `exit`.

```
command = str(input('Enter command: '))
while command != 'exit':
    print('Command: ', command)
    command = str(input('Enter command: '))

print('Program terminated!')
```

**Result:**

```
Enter command: cmd1
Command:  cmd1
Enter command: cmd2
Command:  cmd2
Enter command: exit
Program terminated!
```



## Ατέρμονα Loops

Στην περίπτωση όπου η συνθήκη επανάληψης είναι πάντα αληθής οδηγείστε σε ατέρμονες επαναλήψεις όπου ουσιαστικά το πρόγραμμα σας δεν τερματίζει ποτέ!

Αν τυχόν το πρόγραμμα σας φτάσει σε ατέρμονο loop και δεν τερματίζει τότε μπορείτε να διακόψετε την λειτουργία του πατώντας CTRL+C ή να κλείσετε το παράθυρο εκτέλεσης του Python Shell.

```
While True:  
    print ('Infinite Loop!')
```

**Result:**

```
Infinite Loop!  
Infinite Loop!  
Infinite Loop!  
...
```

## Εργαστηριακές Ασκήσεις

1. Γράψτε ένα πρόγραμμα το οποίο χρησιμοποιεί βρόγχο while και υπολογίζει ακριβώς ότι και ο πιο κάτω κώδικας.

```
for i in range(0,25,4):  
    print(i+1)
```

2. Γράψτε ένα πρόγραμμα το οποίο υπολογίζει το άθροισμα S των τετραγώνων των πολλαπλασίων του 5, μέχρι αυτό να ξεπεράσει την τιμή 5000. Το πρόγραμμα θα πρέπει να εμφανίζει την τελική τιμή του S που έχει υπολογιστεί καθώς και το τελευταίο πολλαπλάσιο του 5 που χρησιμοποιήθηκε στο άθροισμα. Το πρόγραμμά σας θα πρέπει να χρησιμοποιεί βρόγχο while.

$$S = 5^2 + 10^2 + 15^2 + \dots$$

### Παράδειγμα εκτέλεσης

Το άθροισμα S είναι ίσο με 5100

Το τελευταίο πολλαπλάσιο του 5 υψωμένο στο τετράγωνο είναι 40

3. Γράψτε ένα πρόγραμμα που ζητά από το χρήστη ακέραιους αριθμούς και να θυμάται (αποθηκεύει σε μεταβλητές) τον μεγαλύτερο και τον μικρότερο αριθμό εισόδου. Το πρόγραμμα να τερματίζει αν ο χρήστης εισάγει τον αριθμό -1 και εκτυπώνει τους δύο αριθμούς.

### Παράδειγμα εκτέλεσης (με πράσινο η είσοδος του χρήστη)

```
Give a number: 1  
Give a number: 5  
Give a number: 6  
Give a number: 8  
Give a number: -1  
1 is the smallest  
8 is the largest
```



4. Να γράψετε ένα πρόγραμμα που να υπολογίζει το μέγιστο αριθμό και τη θέση του σε μια σειρά θετικών ακέραιων αριθμών απροσδιορίστου μεγέθους που δίνει ο χρήστης (δεν ξέρουμε εκ των προτέρων πόσους αριθμούς θα δώσει ο χρήστης) από το πληκτρολόγιο καθώς επίσης και το μέγεθος που έχει τελικά η σειρά. Η εισαγωγή ακέραιων αριθμών τελειώνει όταν ο χρήστης δώσει 0 ή οποιονδήποτε αρνητικό ακέραιο αριθμό (ο αριθμός αυτός δεν πρέπει να θεωρηθεί μέλος της σειράς). Θεωρήστε ότι ο χρήστης δίνει πάντοτε θετικούς αριθμούς (το πρόγραμμα δε χρειάζεται να κάνει τον έλεγχο).

Παράδειγμα εκτέλεσης (με πράσινο η είσοδος του χρήστη)

Tha dineis thetikoy's akeraious arithmous, ena-ena tin fora.

Gia na stamatiseis dwse to 0 h arnitiko arithmo.

Dwse arithmo 1: 3

Dwse arithmo 2: 45

Dwse arithmo 3: 12

Dwse arithmo 4: 6

Dwse arithmo 5: 9

Dwse arithmo 6: 14

Dwse arithmo 7: 2

Dwse arithmo 8: 34

Dwse arithmo 9: 7

Dwse arithmo 10: 8

Dwse arithmo 11: 0

O megistos einai o 45 kai einai o 2os arithmos mias seiras megethous 10

5. Να γράψετε ένα πρόγραμμα το οποίο αρχικά θα δημιουργεί ένα τυχαίο αριθμό από 1-9. Χρησιμοποιείτε τις πιο κάτω εντολές στην αρχή του προγράμματος για να παράξετε τον τυχαίο αριθμό:

```
import random
number = random.randint(1,9)
```

Στη συνέχεια θα ζητά από τον χρήστη να μαντέψει τον τυχαίο αριθμό και θα τον ενημερώνει εάν ο αριθμός που μάντεψε είναι μικρότερος, μεγαλύτερος ή ο ίδιος με τον τυχαίο αριθμό. Το παιχνίδι τελειώνει όταν ο χρήστης γράψει την λέξη "exit" ή όταν ο χρήστης μαντέψει σωστά τον τυχαίο αριθμό. Επίσης, θα πρέπει να κρατάτε τον συνολικό αριθμό από μαντεπιές και να τον εμφανίζεται όταν το παιχνίδι τελειώσει.

Παράδειγμα Εκτέλεσης (με πράσινο η είσοδος του χρήστη)

Please enter a number from 1 to 9, or type 'exit' to exit the game: 8  
Too high!

Please enter a number from 1 to 9, or type 'exit' to exit the game: 4  
Too high!

Please enter a number from 1 to 9, or type 'exit' to exit the game: 2



Too low!

Please enter a number from 1 to 9, or type 'exit' to exit the game: **3**

You got it!

And it only took you 4 attempts

6. Να γράψετε ένα πρόγραμμα το οποίο διαβάζει ένα θετικό ακέραιο αριθμό από τον χρήστη, να τον αντιστρέφει και να τον τυπώνει.

Παράδειγμα Εκτέλεσης (με πράσινο η είσοδος του χρήστη)

Δώσε ένα θετικό ακέραιο αριθμό: **76542**

Αντίστροφος: 24567

7. Ποιος ακέραιος αριθμός μεταξύ 1 και 10000 έχει το μεγαλύτερο αριθμό διαιρετών, και πόσους διαιρέτες έχει; Γράψτε ένα πρόγραμμα που βρίσκει την απάντηση και να τυπώνει το αποτέλεσμα. Υπάρχει περίπτωση, πολλοί ακέραιοι αριθμοί στο διάστημα 1-10000, να έχουν το ίδιο μέγιστο αριθμό διαιρετών. Το πρόγραμμά σας να τυπώνει μόνο έναν εξ' αυτών.

Παράδειγμα εκτέλεσης

The maximum number of divisors is 64

A number with 64 divisors is 7560

Επίσης σωστή απάντηση μπορεί να είναι και η πιο κάτω.

The maximum number of divisors is 64

A number with 64 divisors is 9240

8. Να γράψετε ένα πρόγραμμα που να υπολογίζει και να εμφανίζει στον οθόνη τον πίνακα της προπαίδειας (πίνακας πολλαπλασιασμού) όπως αυτός φαίνεται πιο κάτω:

|    | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  |
|----|----|----|----|----|----|----|----|----|----|-----|
| 1  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10  |
| 2  | 2  | 4  | 6  | 8  | 10 | 12 | 14 | 16 | 18 | 20  |
| 3  | 3  | 6  | 9  | 12 | 15 | 18 | 21 | 24 | 27 | 30  |
| 4  | 4  | 8  | 12 | 16 | 20 | 24 | 28 | 32 | 36 | 40  |
| 5  | 5  | 10 | 15 | 20 | 25 | 30 | 35 | 40 | 45 | 50  |
| 6  | 6  | 12 | 18 | 24 | 30 | 36 | 42 | 48 | 54 | 60  |
| 7  | 7  | 14 | 21 | 28 | 35 | 42 | 49 | 56 | 63 | 70  |
| 8  | 8  | 16 | 24 | 32 | 40 | 48 | 56 | 64 | 72 | 80  |
| 9  | 9  | 18 | 27 | 36 | 45 | 54 | 63 | 72 | 81 | 90  |
| 10 | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 |